EXPERIMENT-15:

Implement Iris Flower Classification using Naive Bayes classifier.

Code:

```
# ============================================================
# Iris Flower Classification using Naive Bayes
# ============================================================
# Step 1: Upload Dataset
from google.colab import files
uploaded = files.upload()
# Step 2: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 3: Load Dataset
df = pd.read_csv("Iris.csv")
print("First 5 Rows:")
print(df.head())
print("\nDataset Shape:", df.shape)
# Step 4: Remove Id Column (if present)
if "Id" in df.columns:
    df.drop("Id", axis=1, inplace=True)
# Step 5: Encode Target Labels
encoder = LabelEncoder()
df["Species"] = encoder.fit_transform(df["Species"])
print("\nEncoded Dataset:")
print(df.head())
```

```
# Step 6: Split Features and Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
# Step 7: Train Gaussian Naive Bayes Model
model = GaussianNB()
model.fit(X_train, y_train)
# Step 8: Prediction
y_pred = model.predict(X_test)
# Step 9: Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy: {:.2f}%".format(accuracy * 100))
# Step 10: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=encoder.classes_,
    yticklabels=encoder.classes_
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
# Step 11: Classification Report
print("\nClassification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=encoder.classes_
))
# Step 12: Predict New Flower
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)
print("Predicted Flower Species:",
      encoder.inverse_transform(prediction)[0])
```

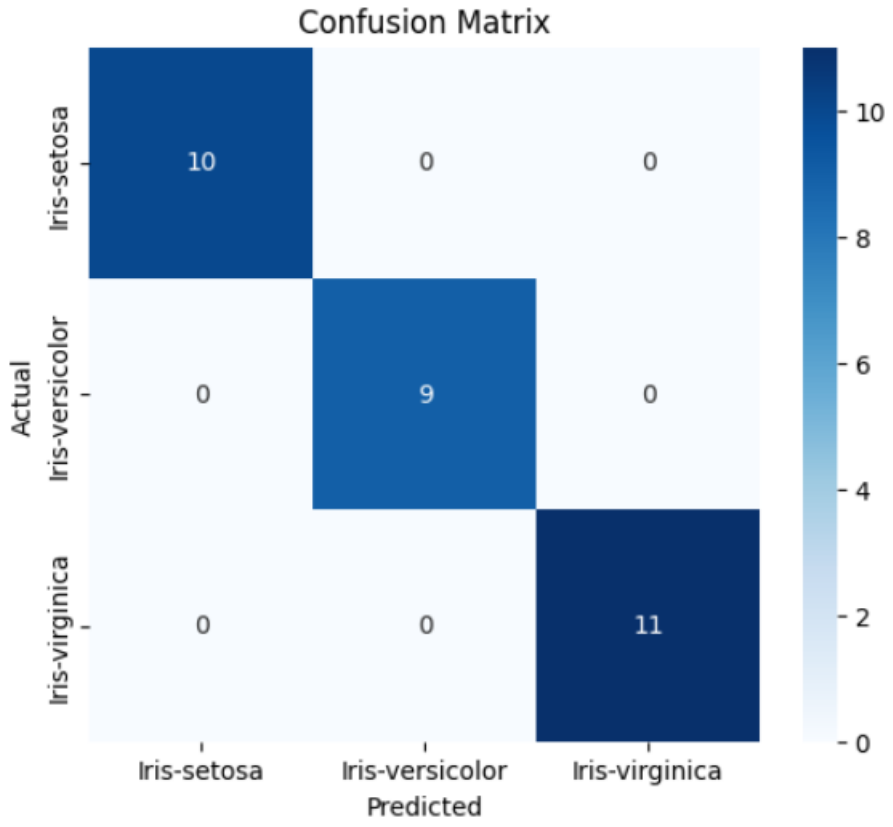
Output:

```
Choose Files  Iris[1].csv
Iris[1].csv(text/csv) - 5107 bytes, last modified: 8/5/2026 - 100% done
Saving Iris[1].csv to Iris[1].csv
First 5 Rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

Dataset Shape: (150, 6)

Encoded Dataset:
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0            5.1           3.5            1.4           0.2        0
1            4.9           3.0            1.4           0.2        0
2            4.7           3.2            1.3           0.2        0
3            4.6           3.1            1.5           0.2        0
4            5.0           3.6            1.4           0.2        0

Accuracy: 100.00%
```




```
Classification Report:

                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

      accuracy                           1.00        30
     macro avg       1.00      1.00      1.00        30
  weighted avg       1.00      1.00      1.00        30

Predicted Flower Species: Iris-setosa
```